

PRACTICAL LAB: JAVASCRIPT OPERATORS MASTERY

Course: DFK20013 Web Design Technology

Topic: 3.1.1 Variables, Data Types, and Operators

Duration: 60 – 90 Minutes

Student Name	MOHAMAD HAIKAL ISMAL BIN ABDUL AZIZ
Registration No	10DIT25F1089
Date	3/3/2026

1. Learning Outcomes

By the end of this lab, students should be able to:

- Implement **Arithmetic Operators** for calculations.
- Use **Assignment Operators** to update values efficiently.
- Control application states using **Increment/Decrement**.
- Validate user access using **Relational** and **Logical** operators.

2. Practical Task: The All-in-One Operator Workshop

Create a file named lab_operators.html. Copy and complete the coding below.

HTML

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>DFK20013: JS Operators Workshop</title>

    <style>

        body { font-family: 'Segoe UI', sans-serif; max-width: 850px;
margin: 2rem auto; background-color: #f0f2f5; padding: 20px; color: #333;
}

        section { background: white; padding: 25px; border-radius: 12px;
margin-bottom: 25px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }

        h2 { color: #2c3e50; border-left: 5px solid #3498db; padding-
left: 10px; margin-top: 0; }

        .output-box { background: #2c3e50; color: #27ae60; padding: 15px;
border-radius: 6px; font-family: 'Courier New', monospace; margin-top:
10px; }
```

```

        .short-note { background: #fff3cd; color: #856404; padding: 15px;
border-radius: 6px; margin-top: 15px; border-left: 5px solid #ffc107;
display: none; font-size: 0.9rem; }

        input { padding: 8px; width: 100px; border-radius: 4px; border:
1px solid #ccc; }

        button { padding: 8px 20px; background: #3498db; color: white;
border: none; border-radius: 4px; cursor: pointer; font-weight: bold; }

        button:hover { background: #2980b9; }

        code { background: #eee; padding: 2px 4px; border-radius: 3px;
font-weight: bold; color: #d63384; }

    </style>

```

```
</head>
```

```
<body>
```

```

    <header style="text-align: center; margin-bottom: 30px;">

```

```

        <h1>JavaScript Operators Mastery</h1>

```

```

        <p>Follow the steps and observe the Console and UI outputs.</p>

```

```
    </header>
```

```
<section>
```

```

    <h2>Task 1: Basic Arithmetic (Check Console)</h2>

```

```

    <p>This task demonstrates background calculations using
<code>+</code>, <code>-</code>, <code>*</code>, <code>/</code>,
<code>*</code>, and <code>%</code>.</p>

```

```

    <button onclick="runTask1()">Run Console Logic</button>

```

```
</section>
```

```
<section>
```

```

    <h2>Task 2: POS System (Arithmetic & Assignment)</h2>

```

```

    <p>Price: RM 50.00. <strong>Promo:</strong> Buy > 4 units to get
RM 20.00 discount.</p>

```

```

    Quantity: <input type="number" id="qty" value="1">

```

```

    <button onclick="runTask2()">Calculate Total</button>

```

```

    <div id="outPOS" class="output-box">Final Total: RM 0.00</div>

```

```

    <div id="notePOS" class="short-note"></div>

```

```
</section>
```

```
<section>
```

```

    <h2>Task 3: Stock Control (Increment & Decrement)</h2>

```

```

        <p>Simulating inventory updates.</p>

        Stock Level: <span id="stockVal" style="font-size: 1.5rem; font-
weight: bold;">10</span><br><br>

        <button onclick="runTask3('inc')">Add (+)</button>

        <button onclick="runTask3('dec') "
style="background:#6c757d;">Remove (-)</button>

        <div id="noteStock" class="short-note"></div>

    </section>

<section>

    <h2>Task 4: Gate Access (Relational & Logical)</h2>

    <p>Rules: Age 18+ OR (Age 13+ AND Parent present).</p>

    Age: <input type="number" id="age" placeholder="Age">

    <label><input type="checkbox" id="parent"> With
Parent?</label><br><br>

    <button onclick="runTask4()" style="background:#27ae60;">Verify
Entry</button>

    <div id="outSecurity" class="output-box">Status: Waiting...</div>

    <div id="noteSecurity" class="short-note"></div>

</section>

```

Inside the <script> tag, type all the following code:

JAVASCRIPT:

```

1. <script>
    // Helper to show explanations
    function showNote(id, text) {
        const noteDiv = document.getElementById(id);
        noteDiv.style.display = 'block';
        noteDiv.innerHTML = text;
    }

```

2.	<pre> // --- TASK 1: ARITHMETIC (BASIC) --- function runTask1() { console.log("--- Task 1: Arithmetic Operations ---"); let students = 31; students = students + 1; // Addition students = students - 2; // Subtraction students = students * 2; // Multiplication students = students / 2; // Division let power = 10 ** 2; // Exponentiation (100) let remainder = 31 % 2; // Modulus (1 - Odd) console.log("Final Student Count: " + students); console.log("10 to the power of 2: " + power); console.log("Remainder of 31/2: " + remainder); alert("Check Browser Console (F12) for results!"); } </pre>	
3.	<pre> // --- TASK 2: POS SYSTEM (ARITHMETIC & ASSIGNMENT) --- function runTask2() { let price = 50; let qty = parseInt(document.getElementById('qty').value); // a. Arithmetic Operator (*) let total = price * qty; let note = "Subtopic (a) Arithmetic: <code>*</code> multiplied " + price + " by " + qty + "
"; // b. Assignment Operator (-=) if (qty > 4) { total -= 20; // Shorthand for total = total - 20 note += "Subtopic (b) Assignment: <code>-=</code> updated the total by subtracting the discount."; } document.getElementById('outPOS').textContent = "Final Total: RM " + total.toFixed(2); showNote('notePOS', note); } </pre>	

4.

```
// --- TASK 3: INCREMENT & DECREMENT ---
let stock = 10;
function runTask3(type) {
    let note = "";
    if (type === 'inc') {
        stock++; // (c) Increment
        note = "<b>Subtopic (c) Increment:</b> <code>++</code> increased the stock by 1.";
    } else {
        stock--; // (c) Decrement
        note = "<b>Subtopic (c) Decrement:</b> <code>--</code> decreased the stock by 1.";
    }
    document.getElementById('stockVal').textContent = stock;
    showNote('noteStock', note);
}
```

5.

```
// --- TASK 4: RELATIONAL & LOGICAL ---
function runTask4() {
    let age = parseInt(document.getElementById('age').value);
    let hasParent = document.getElementById('parent').checked;
    let status = document.getElementById('outSecurity');

    if (isNaN(age)) return alert("Please enter a valid age.");

    // d. Relational: >=
    // e. Logical: || (OR), && (AND)
    let isEligible = (age >= 18) || (age >= 13 && hasParent);

    status.textContent = isEligible ? "Status: ACCESS GRANTED  
✔" : "Status: ACCESS DENIED ✖";
    status.style.color = isEligible ? "#27ae60" : "#ff4d4d";

    let note = "<b>Subtopic (d) Relational:</b> <code>>=</code> compared Age (" + age + ") to the limit.<br>";
    note += "<b>Subtopic (e) Logical:</b> Used <code>||</code> (OR) and <code>&&</code> (AND) to combine rules.";
    showNote('noteSecurity', note);
}
</script>
```

Capture a screenshot of the output from the previous steps. **Observe** the results and **explain** your understanding of the process.

1. Screenshot of the output

The screenshot shows the Chrome DevTools interface. The 'Elements' panel displays the HTML document structure. The root is `<!DOCTYPE html>`, followed by `<html lang="en">`. Inside the `<head>` are `<meta charset="UTF-8">`, `<title>DFK20013: JS Operators Workshop</title>`, and `<style>...</style>`. The `<body>` contains a navigation bar, a header with centered text, and several sections. The 'Styles' panel shows the default browser styles for the `body` element, including background color, font family, and text alignment.

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>DFK20013: JS Operators Workshop</title>
    <style>...</style>
  </head>
  <body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">...</nav>
    <header style="text-align: center; margin-bottom: 30px;">...</header>
    <section>...</section>
    <section>...</section>
    <section>...</section>
    <section>...</section>
    <script>...</script>
  </body>
</html>
  
```

```

body {
  background-color: #f4f6f9;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
  margin: 0;
  font-family: var(--bs-body-font-family);
  font-size: var(--bs-body-font-size);
  font-weight: var(--bs-body-font-weight);
  line-height: var(--bs-body-line-height);
  color: var(--bs-body-color);
  text-align: var(--bs-body-text-align);
  background-color: var(--bs-body-bg);
  -webkit-text-size-adjust: 100%;
  -webkit-tap-highlight-color: transparent;
}
  
```

Explanation

This task demonstrates how JavaScript handles mathematical operations like addition, subtraction, multiplication, and division. It also uses the modulus operator (%) to find remainders and exponentiation (**) for power calculations.

2 Screenshot of the output

Task 2: POS System (Arithmetic & Assignment)

Price: RM 50.00. **Promo:** Buy > 4 units to get RM 20.00 discount.

Quantity:

Calculate Total

Final Total: RM 150.00

Subtopic (a) Arithmetic: * multiplied 50 by 3.

Task 2: POS System (Arithmetic & Assignment)

Price: RM 50.00. **Promo:** Buy > 4 units to get RM 20.00 discount.

Quantity:

Calculate Total

Final Total: RM 230.00

Subtopic (a) Arithmetic: * multiplied 50 by 5.

Subtopic (b) Assignment: -= updated the total by subtracting the discount.

Explanation

This section calculates the total price by multiplying quantity and unit price. It uses the **subtraction assignment operator** (`-=`) to automatically deduct a discount from the total if the quantity exceeds 4 units.

3 .	Screenshot of the output Task 3: Stock Control (Increment & Decrement) Simulating inventory updates. Stock Level: 15 Add (+) Remove (-) Subtopic (c) Increment: ++ increased the stock by 1. Task 3: Stock Control (Increment & Decrement) Simulating inventory updates. Stock Level: 5 Add (+) Remove (-) Subtopic (c) Decrement: -- decreased the stock by 1.
	Explanation This task simulates inventory management by using the increment (++) operator to add items to the stock and the decrement (--) operator to remove them.

4 Screenshot of the output

.

Task 4: Gate Access (Relational & Logical)

Rules: Age 18+ OR (Age 13+ AND Parent present).

Age:



With Parent?

Verify Entry

Status: ACCESS GRANTED ☒

Subtopic (d) Relational: `>=` compared Age (19) to the limit.

Subtopic (e) Logical: Used `||` (OR) and `&&` (AND) to combine rules.

Task 4: Gate Access (Relational & Logical)

Rules: Age 18+ OR (Age 13+ AND Parent present).

Age:

☐

With Parent?

Verify Entry

Status: ACCESS DENIED ✖

Subtopic (d) Relational: `>=` compared Age (17) to the limit.

Subtopic (e) Logical: Used `||` (OR) and `&&` (AND) to combine rules.

Task 4: Gate Access (Relational & Logical)

Rules: Age 18+ OR (Age 13+ AND Parent present).

Age:

☒

With Parent?

Verify Entry

Status: ACCESS GRANTED ✔

Subtopic (d) Relational: `>=` compared Age (17) to the limit.

Subtopic (e) Logical: Used `||` (OR) and `&&` (AND) to combine rules.

Explanation

This logic verifies entry eligibility using relational operators (like `>=`) to check age. It combines multiple conditions using logical operators such as `&&` (AND) and `||` (OR) to determine if a user can enter based on age or parental presence.

3. Quick Check (Reflection)

Instruction: Answer the following questions based on your coding results.

1. **Modulus Utility:** Based on Task 1, if `students % 2` returns 0, is the number even or odd?

The modulus operator (%) is used to determine if a number is even or odd by checking the remainder when divided by 2.

2. **Shorthand Power:** In Task 2, rewrite `total -= 20` using the standard assignment method.

The expression `total -= 20` is a shorthand for `total = total - 20`.

3. **Logical Priority:** In Task 4, why did we put `(age >= 13 && hasParent)` in parentheses?

Parentheses () are used in Task 4 to group the age and parental consent conditions together, ensuring they are evaluated as a single unit before being compared to the adult age limit.

4. **Time Calculation:** (Extra Challenge) If a variable `totalMinutes = 130`, write the formula to find the remaining minutes after converting to hours using the **Modulus** operator.

To calculate the remaining minutes after converting to hours, the formula is `remainingMinutes = totalMinutes % 60`;